

Automation Framework For SDET

Part 1: Core Design & Structure

★ Topics 01 to 12 — Foundation to Reporting ★



Kalpesh Jain

SDET @ Algebrik AI | Mentor | 42K+ Community ■

Bengaluru, Karnataka, India

[linkedin.com/in/kalpeshnjain09](https://www.linkedin.com/in/kalpeshnjain09)

JSPM University Pune

Intro to Framework	Types of Frameworks	Framework Architecture	Page Object Model
Project Structure	Config Management	Driver Factory	Base Test
Utilities & Helpers	Test Data Management	Logging	Reporting

Introduction

This is **Part 1** of the Automation Framework For SDET Complete Guide. It covers **Topics 01–12** — the core framework design and structure topics asked in every SDET interview.

Complete Part 1 before moving to Part 2 (Execution, CI/CD & Best Practices).

Topic Group	What You Will Learn
Introduction & Types	What is a framework, why it matters, framework types
Architecture & POM	Framework layers, POM design pattern, execution flow
Project Structure & Config	Folder organization, environment management
Driver Factory & Base Test	Browser management, shared test setup
Utilities, Data & Reports	Helpers, test data, logging, and reporting

Table of Contents

PART 1 — CORE DESIGN & STRUCTURE (Topics 01–12)

- 01. Introduction to Automation Framework ★
- 02. Types of Automation Frameworks ★
- 03. Framework Architecture ★
- 04. Page Object Model (POM) ★
- 05. Project Structure ★
- 06. Configuration Management ★
- 07. Driver Factory ★
- 08. Base Test ★
- 09. Utilities and Helper Classes ★
- 10. Test Data Management ★
- 11. Logging ★
- 12. Reporting ★

INTRODUCTION TO AUTOMATION FRAMEWORK

01. Introduction to Automation Framework ★

► What is an Automation Framework?

What is it: An Automation Framework is a structured collection of guidelines, reusable components, libraries, utilities, and standards used to build, execute, and maintain automated test scripts efficiently. It provides a common architecture so that automation tests are scalable, maintainable, and reusable.

Key Components:

- Reusable code
- Standard project structure
- Test execution flow
- Configuration management
- Reporting
- Logging

How YOU should answer: An Automation Framework is a reusable architecture that organizes automation code, test data, utilities, reporting, and execution flow to make test automation scalable, maintainable, and easy to manage.

Example

Instead of writing browser launch code in every test, the framework provides a common Base Test class that handles browser setup automatically.

► Why Automation Framework is Important

What is it: An Automation Framework standardizes automation development and improves collaboration across teams.

Key Components:

- Reusability
- Scalability
- Maintainability
- Faster execution
- Reduced code duplication

How YOU should answer: Automation Frameworks reduce maintenance effort, improve code quality, and make automation projects easier to scale as applications grow.

Example

A login utility created once can be reused across hundreds of test cases.

Common Mistakes

- ✗ Thinking a framework is just a collection of test scripts.
- ✗ Creating duplicate utility methods across multiple files.
- ✗ Writing independent scripts without reusable components.

- ✗ Hardcoding locators and test data.

Expected Interview Questions

- What is an Automation Framework?
- Why do we need an Automation Framework?
- What problems does it solve?
- Why is an Automation Framework needed?
- What are its advantages?

★ Final Interview Gold

1. - What is an Automation Framework
2. - Goals of an Automation Framework
3. - Core Components
4. - Benefits
5. - Real-Time Framework Flow
6. - Best Practices
7. - Framework Architecture Overview
8. Automation Framework for SDET

TYPES OF AUTOMATION FRAMEWORKS

02. Types of Automation Frameworks ★

► What are Automation Framework Types?

What is it: Automation Frameworks can be designed in different ways depending on the application's complexity, team size, and project requirements. Each framework type focuses on improving code organization, reusability, and maintainability.

Key Components:

- Different framework architectures
- Different levels of reusability
- Different maintenance efforts
- Different scalability

How YOU should answer: Automation Frameworks are categorized into different types based on how automation code, test data, and reusable components are organized.

Example

Small Startup Project

```
↓  
Simple Modular Framework  
Enterprise Banking Project  
↓  
Hybrid Framework
```

► Why Framework Types are Important

What is it: Choosing the right framework directly impacts project scalability and maintenance.

Key Components:

- Project size
- Team collaboration
- Maintenance
- Reusability

How YOU should answer: Different projects require different framework designs. Selecting the appropriate framework reduces maintenance effort and improves long-term scalability.

```
Example  
Simple Website  
↓  
Modular Framework  
Large Enterprise Product  
↓  
Hybrid Framework
```

► Linear Framework

What is it: Linear Framework is the simplest automation framework where test scripts are written sequentially without reusable components.

Key Components:

- Sequential scripts
- No reusability
- Easy to develop
- High maintenance

How YOU should answer: Linear Framework is suitable only for very small projects because each test case contains its own automation logic.

```
Example  
Login Test  
↓  
Search Test  
↓
```

```
Checkout Test
```

```
(All scripts written separately)
```

► Modular Framework

What is it: The application is divided into independent modules, and each module has its own automation scripts.

Key Components:

- Module-based design
- Reusable modules
- Better maintenance

How YOU should answer: Modular Framework divides the application into smaller independent modules, making automation easier to maintain.

```
Example
```

```
Login Module
```

```
Dashboard Module
```

```
Profile Module
```

```
Payment Module
```

► Data-Driven Framework

What is it: Test data is separated from automation scripts. The same test script executes with multiple datasets.

Key Components:

- Excel
- CSV
- JSON
- Database
- Multiple datasets

How YOU should answer: Data-Driven Framework separates test logic from test data, allowing one script to execute multiple test scenarios.

```
Example
```

```
Login Script
```

```
↓
```

```
Username1 Password1
```

```
Username2 Password2
```

```
Username3 Password3
```

► Keyword-Driven Framework

What is it: Automation actions are controlled using predefined keywords.

Key Components:

- Keywords
- Action tables
- Excel-driven execution

How YOU should answer: Keyword-Driven Framework executes automation based on predefined keywords such as Click, Enter Text, and Verify.

Example

```
Open Browser
Click Login
Enter Username
Click Submit
```

Common Mistakes

- ✗ Thinking every company uses the same framework.
- ✗ Choosing an overly complex framework for a small project.

Expected Interview Questions

- What are Automation Framework types?
- Why do different framework types exist?
- Why is framework selection important?
- Which framework is most commonly used?
- What is Linear Framework?

★ Final Interview Gold

1. - Types of Automation Frameworks
2. - Linear Framework
3. - Modular Framework
4. - Data-Driven Framework
5. - Keyword-Driven Framework
6. - Hybrid Framework
7. - BDD Framework
8. - Framework Comparison

FRAMEWORK ARCHITECTURE

03. Framework Architecture ★

► What is Framework Architecture?

What is it: Framework Architecture is the overall design and organization of an automation framework. It defines how different components interact to execute automated test cases efficiently.

Key Components:

- Structured design
- Layered architecture
- Separation of responsibilities
- Reusable components
- Maintainable code

How YOU should answer: Framework Architecture is the blueprint of an automation framework that defines how test scripts, page objects, utilities, configuration files, reporting, and execution components work together.

Example

Test Scripts

↓

Page Objects

↓

Utilities

↓

Playwright API

↓

► **Why Framework Architecture is Important**

What is it: A well-designed architecture improves scalability, maintainability, readability, and collaboration.

Key Components:

- Easy maintenance
- Scalability
- Reusability
- Team collaboration
- Standardization

How YOU should answer: A good framework architecture reduces code duplication, improves maintainability, and enables multiple team members to work efficiently.

Example

New feature added

↓

Only Page Object updated

↓

Existing test cases continue working

Common Mistakes

- ✗ Mixing business logic with framework code.
- ✗ No separation of reusable components.
- ✗ Keeping all automation code in one folder.
- ✗ Directly interacting with Playwright APIs inside test scripts.

Expected Interview Questions

- What is Framework Architecture?
- Why is framework architecture important?
- What problems does it solve?
- What are the layers in your framework?
- Why separate utilities from test scripts?

★ Final Interview Gold

1. - Framework Architecture
2. - Framework Layers
3. - Folder Structure
4. - Framework Execution Flow
5. - Responsibilities of Each Layer
6. - Enterprise Architecture
7. - Framework Best Practices
8. Page Object Model (POM)

PAGE OBJECT MODEL (POM)

04. Page Object Model (POM) ★

► What is Page Object Model (POM)?

What is it: Page Object Model (POM) is a design pattern used in automation testing where each web page is represented as a separate class. Each page class contains: - Web locators - Page actions - Business methods This separates UI interactions from test logic.

Key Components:

- Page Classes
- Locators
- Action Methods
- Business Functions
- Reusable Components

How YOU should answer: Page Object Model is a design pattern that separates page-specific operations from test scripts by creating a dedicated class for each application page.

Example

Login Test

↓

LoginPage

↓

login(username, password)

↓

DashboardPage

► Why POM is Important

What is it: POM improves code organization and reduces maintenance effort.

Key Components:

- Reusability
- Maintainability
- Scalability
- Cleaner code

How YOU should answer: POM separates business logic from UI interactions, making automation easier to maintain and scale.

Example

Login button locator changes

↓

Update only LoginPage

↓

All tests continue working

Common Mistakes

- ✗ Writing locators directly inside test scripts.
- ✗ Mixing assertions with page classes.
- ✗ Duplicating locators across multiple tests.
- ✗ Calling Playwright APIs directly from test classes.

Expected Interview Questions

- What is POM?
- Why do we use POM?
- Why is POM important?
- How does POM reduce maintenance?
- What are the components of POM?

★ Final Interview Gold

1. - What is Page Object Model
2. - POM Architecture
3. - Components of POM
4. - Advantages of POM
5. - POM vs Traditional Automation
6. - Real-Time Playwright Framework
7. - POM Best Practices
8. Project Structure

PROJECT STRUCTURE

05. Project Structure ★

► What is Project Structure?

What is it: Project Structure is the logical organization of files and folders within an automation framework. A well-designed project structure improves readability, maintainability, scalability, and collaboration among team members.

Key Components:

- Organized folders
- Separation of responsibilities
- Reusable components
- Easy navigation
- Standardized framework layout

How YOU should answer: Project Structure defines how automation files, folders, utilities, configurations, reports, and test scripts are organized to build a scalable and maintainable automation framework.

Example

```
automation-framework/  
tests/  
pages/  
utils/  
config/  
reports/
```

► Why Project Structure is Important

What is it: A proper project structure allows teams to easily locate files, maintain automation scripts, and extend the framework.

Key Components:

- Better organization
- Faster maintenance
- Easy scalability
- Improved collaboration

How YOU should answer: A proper project structure reduces confusion, improves collaboration, and keeps the automation framework scalable as the application grows.

Example

New module added

↓

Create new Page Object

↓

No changes to existing modules

Common Mistakes

- ✗ Keeping every file inside one folder.
- ✗ Mixing reusable code with test scripts.
- ✗ Creating random folders without standards.
- ✗ Storing reports and screenshots together with source code.

Expected Interview Questions

- What is Project Structure?
- Why is it important?
- Why do we need a proper project structure?
- What does the utils folder contain?
- Why separate test-data from tests?

★ Final Interview Gold

1. - Project Structure
2. - Standard Folder Organization
3. - Folder Responsibilities
4. - Framework Execution Flow
5. - Enterprise Project Structure
6. - CI/CD Integration
7. - Project Structure Best Practices
8. Configuration Management

CONFIGURATION MANAGEMENT

06. Configuration Management ★

► What is Configuration Management?

What is it: Configuration Management is the process of storing and managing application settings, environment-specific values, credentials, URLs, browser settings, and other configurable data separately from automation code. Instead of hardcoding values inside test scripts, frameworks read them from configuration files.

Key Components:

- Environment URLs
- Browser Configuration
- Credentials
- Timeouts
- API Endpoints
- Database Details

How YOU should answer: Configuration Management is the practice of externalizing application settings from automation code, allowing the same framework to execute across multiple environments without code changes.

Example

Environment = QA

↓

Framework reads

Base URL

Username

Password

Browser

↓

► Why Configuration Management is Important

What is it: Configuration Management makes automation frameworks flexible and reusable across multiple environments.

Key Components:

- Reusability
- Environment Independence
- Security
- Easy Maintenance
- Scalability

How YOU should answer: Configuration Management eliminates hardcoded values, improves security, and allows the framework to run against multiple environments using the same codebase.

Example

Today

QA Environment

Tomorrow

Production Environment

↓

Only configuration changes

No code changes

Common Mistakes

- ✗ Hardcoding URLs inside tests.
- ✗ Hardcoding usernames and passwords.
- ✗ Maintaining separate projects for each environment.
- ✗ Committing sensitive credentials to GitHub.

Expected Interview Questions

- What is Configuration Management?
- Why is it important?
- Why should configuration be externalized?
- Which configuration files have you used?
- Where do you store URLs?

★ Final Interview Gold

1. - Configuration Management
2. - Environment Management
3. - Configuration Files
4. - Config Reader
5. - Configuration Flow
6. - Enterprise Configuration
7. - Configuration Best Practices
8. Driver Factory

DRIVER FACTORY

07. Driver Factory ★

► What is Driver Factory?

What is it: Driver Factory is a reusable framework component responsible for creating, managing, and closing browser instances. Instead of launching browsers inside every test case, the framework delegates browser management to the Driver Factory.

Key Components:

- Browser initialization
- Browser closing
- Browser configuration
- Browser selection
- Driver lifecycle management

How YOU should answer: Driver Factory is a centralized component responsible for creating and managing browser instances. It provides reusable browser initialization logic across the automation framework.

Example

Run Test

↓

Driver Factory

↓

Launch Chrome

↓

Return Browser Instance

↓

► Why Driver Factory is Important

What is it: Driver Factory centralizes browser management and improves framework maintainability.

Key Components:

- Code reusability
- Easy maintenance
- Cross-browser support
- Better scalability

How YOU should answer: Driver Factory removes duplicate browser initialization code and provides a single place to manage browser creation and configuration.

Example

Chrome

Firefox

Edge



One Driver Factory

Common Mistakes

- ✗ Launching browsers directly inside test scripts.
- ✗ Creating multiple browser initialization methods.
- ✗ Browser launch code copied into multiple tests.
- ✗ Mixing browser management with test logic.

Expected Interview Questions

- What is Driver Factory?
- Why do we need Driver Factory?
- Why is Driver Factory important?
- What are the responsibilities of Driver Factory?
- How do you support multiple browsers?

★ Final Interview Gold

1. - Driver Factory
2. - Browser Initialization
3. - Driver Factory Architecture
4. - Cross-Browser Execution
5. - Browser Lifecycle Management
6. - Enterprise Driver Factory
7. - Driver Factory Best Practices
8. Base Test

BASE TEST

08. Base Test ★

► What is Base Test?

What is it: Base Test is a parent class in an automation framework that contains common setup and teardown logic shared by all test classes. Instead of writing browser initialization, configuration loading, and cleanup code in every test, all test classes inherit these capabilities from Base Test.

Key Components:

- Test setup
- Browser initialization

- Configuration loading
- Driver management
- Test cleanup
- Shared framework utilities

How YOU should answer: Base Test is a reusable parent class that contains common framework initialization and cleanup logic. Every test class extends the Base Test to avoid code duplication and maintain consistency.

Example

```
LoginTest
↓
extends BaseTest
↓
Browser Launch
↓
Execute Test
↓
```

► Why Base Test is Important

What is it: Base Test centralizes common test initialization and cleanup activities.

Key Components:

- Reusability
- Maintainability
- Standard execution
- Less duplicate code
- Easy framework management

How YOU should answer: Base Test eliminates duplicate setup code by providing a common execution lifecycle for all automated tests.

Example

```
100 Test Classes
↓
One Base Test
↓
Common Browser Setup
```

Common Mistakes

- ✗ Initializing the browser inside every test class.
- ✗ Duplicating setup and teardown methods.
- ✗ Writing setup code repeatedly in every test.
- ✗ Business logic inside Base Test.

Expected Interview Questions

- What is Base Test?
- Why is Base Test used?
- Why do we need Base Test?
- What are the responsibilities of Base Test?
- How does Base Test work with Driver Factory?

★ Final Interview Gold

1. - Base Test
2. - Base Test Responsibilities
3. - Base Test Architecture
4. - Execution Flow
5. - Driver Factory Integration
6. - Enterprise Base Test
7. - Base Test Best Practices
8. Utilities and Helper Classes

UTILITIES AND HELPER CLASSES

09. Utilities and Helper Classes ★

► What are Utilities and Helper Classes?

What is it: Utility Classes are reusable classes that contain common helper methods used throughout the automation framework. Instead of writing the same code repeatedly, Utility Classes centralize common functionalities that can be reused across multiple test cases and page objects.

Key Components:

- Reusable methods
- Common business-independent logic
- Generic helper functions
- Framework support methods
- Shared utilities

How YOU should answer: Utility Classes contain reusable helper methods that perform common framework operations such as reading test data, taking screenshots, generating random data, formatting dates, waiting for elements, and handling files.

Example

Login Test

```
↓  
Take Screenshot  
↓  
Screenshot Utility  
↓  
Save Image
```

► Why Utility Classes are Important

What is it: Utility Classes improve framework reusability and reduce duplicate code.

Key Components:

- Reusability
- Maintainability
- Cleaner framework
- Better organization
- Easy debugging

How YOU should answer: Utility Classes centralize reusable framework functionalities, making automation code cleaner, easier to maintain, and more modular.

```
Example  
  
100 Tests  
↓  
One Screenshot Utility  
↓  
Reusable Everywhere
```

Common Mistakes

- ✗ Writing duplicate helper methods in multiple classes.
- ✗ Keeping utility logic inside test classes.
- ✗ Copying the same helper methods into multiple page classes.
- ✗ Creating one large Utility Class for everything.

Expected Interview Questions

- What are Utility Classes?
- Why do we need Utility Classes?
- Why are Utility Classes important?
- Which Utility Classes have you implemented?
- How do Utility Classes work?

★ Final Interview Gold

1. - Utility Classes
2. - Screenshot Utility

3. - Excel Utility
4. - Random Data Utility
5. - Utility Architecture
6. - Enterprise Utility Structure
7. - Utility Best Practices
8. Test Data Management

TEST DATA MANAGEMENT

10. Test Data Management ★

► What is Test Data Management?

What is it: Test Data Management (TDM) is the process of creating, storing, maintaining, and managing data used for automated testing. Instead of hardcoding values inside test scripts, test data is stored externally and loaded during execution.

Key Components:

- External test data
- Data creation
- Data maintenance
- Data validation
- Data cleanup
- Data security

How YOU should answer: Test Data Management is the process of managing test inputs separately from automation scripts. It improves reusability, maintainability, and enables Data-Driven Testing.

Example

Login Test

↓

Read Data

↓

users.json

↓

Execute Login

↓

► Why Test Data Management is Important

What is it: Proper Test Data Management allows the same automation scripts to execute with multiple datasets.

Key Components:

- Reusability
- Scalability
- Data independence
- Easy maintenance
- Better coverage

How YOU should answer: Separating test data from automation code makes scripts reusable and simplifies maintenance when test inputs change.

Example

One Login Script

↓

100 User Records

↓

100 Test Executions

► **Data-Driven Testing**

What is it: Data-Driven Testing executes the same test multiple times using different datasets.

Example

Login

↓

User1

↓

User2

↓

User3

Common Mistakes

- ✗ Hardcoding usernames and passwords.
- ✗ Keeping test data inside Page Objects.
- ✗ Duplicate test scripts for different data.
- ✗ Mixing test data with framework code.

Expected Interview Questions

- What is Test Data Management?
- Why do we need Test Data Management?
- Why should test data be externalized?
- What are the different types of test data?
- Which test data sources have you worked with?

★ Final Interview Gold

1. - Test Data Management
2. - Data-Driven Testing
3. - Test Data Sources
4. - Dynamic Test Data
5. - Enterprise Test Data Management
6. - JSON vs Excel
7. - Test Data Best Practices
8. Logging

LOGGING

11. Logging ★

► What is Logging?

What is it: Logging is the process of recording important events, actions, errors, warnings, and execution details while automation tests are running. Logs help automation engineers understand what happened during execution and quickly identify failures.

Key Components:

- Execution Logs
- Error Logs
- Warning Logs
- Information Logs
- Debug Logs

How YOU should answer: Logging is the process of capturing execution details during automation. It helps in debugging failures, tracking execution flow, and identifying issues quickly.

Example

Test Started

↓

Browser Launched

↓

Login Successful

↓

Dashboard Loaded

↓

► Why Logging is Important

What is it: Logging provides visibility into framework execution and simplifies troubleshooting.

Key Components:

- Debugging
- Traceability
- Error Analysis
- Faster Issue Resolution
- Execution Monitoring

How YOU should answer: Logging helps track every important execution step, making it easier to identify where and why a test failed.

Example

Click Login

↓

Element Not Found

↓

Error Logged

↓

Framework Continues Cleanup

Common Mistakes

- ✗ Using only `console.log()`.
- ✗ Not logging important execution steps.
- ✗ Logging unnecessary information.
- ✗ Logging everything as `ERROR`.

Expected Interview Questions

- What is Logging?
- Why is Logging important?
- Why do we need Logging?
- Difference between Logging and Reporting?
- Which logs have you used?

★ Final Interview Gold

1. - Logging
2. - Logging vs Reporting
3. - Logging Levels
4. - Logging Frameworks

- 5. - Enterprise Logging
- 6. - Debugging with Logs
- 7. - Logging Best Practices
- 8. Reporting

REPORTING

12. Reporting ★

► What is Reporting?

What is it: Reporting is the process of generating a structured summary of automation test execution. Unlike logs, reports provide a clear overview of test results, execution statistics, failures, screenshots, and execution duration, making them useful for both technical and non-technical stakeholders.

Key Components:

- Test Summary
- Pass/Fail Status
- Execution Time
- Failure Details
- Screenshots
- Environment Information

How YOU should answer: Reporting is the process of generating a structured summary of automation execution, including test results, execution time, failure details, screenshots, and overall execution statistics.

Example

Regression Suite

↓

120 Tests

↓

115 Passed

5 Failed

↓

HTML Report Generated

► Why Reporting is Important

What is it: Reporting provides visibility into automation execution and helps stakeholders quickly understand execution status.

Key Components:

- Execution Summary
- Better Visibility
- Faster Analysis
- Easy Sharing
- Historical Tracking

How YOU should answer: Reports help QA engineers, developers, and managers understand execution results without reading logs.

Example

Regression Complete

↓

HTML Report

↓

Shared with QA Team

Common Mistakes

- ✗ Depending only on console output.
- ✗ Reports without screenshots.
- ✗ Sharing raw logs instead of reports.
- ✗ Generating multiple reports unnecessarily.

Expected Interview Questions

- What is Reporting?
- Why is Reporting important?
- Why do we need Reporting?
- Difference between Reporting and Logging?
- Which reporting tools have you used?

★ Final Interview Gold

1. - Reporting
2. - Reporting vs Logging
3. - Report Components
4. - Reporting Tools
5. - Report Generation Flow
6. - Enterprise Reporting
7. - Reporting Best Practices
8. Screenshots on Failure

Kalpesh Jain | SDET @ Algebrik AI | linkedin.com/in/kalpeshnjain09 | Part 1 of 2